

Unidad 4 | Introducción a la programación web CSS

Expectativas de logro	Contenidos
1. Conocen distintas formas de combinar código CSS con código HTML5	1. Qué es CSS 1.1 Mejor fuera de línea que en línea 1.2 Enlazar estilos CSS 1.3 Etiquetas autocerrables 1.4 Sintaxis 2. Medidas 3. Color de fondo, altura y ancho 4. Bordes 5. Enlaces y "text-decoration" (decoración de texto) 6. Elementos HTML como selectores 6.1 Selectores múltiples 6.2 Un selector para controlar los demás selectores 7. Ramificación 7.1 Clases 7.2 Identificadores 7.4 Selectores de pseudoclase 7.5 Enlaces 7.6 Pseudoclase first-child y nth-child 8. Posicionamiento 9. Layout o bosquejo 9.1 Márgenes 9.2.1 Margen superior, derecho, inferior, izquierdo 9.2 Padding (relleno) 9.2.1 Valores negativos 10. Floats 10.1 Colocar un elemento estático de forma determinada 10.2 Posicionamiento absoluto 10.3 Posicionamiento relativo 10.3 Posicionamiento fijo
2. Saben utilizar las nuevas etiquetas de código CSS aprendidas.	
3. Aprenden como mejorar el estilo de las páginas web usando CSS	
4. Crean páginas web mediante el uso de código HTML5 y CSS.	
5. Ponen en práctica lo aprendido para desarrollar nuevos estilos de páginas web.	
6. Son capaces de crear un sitio web moderno.	

Procesos y actividades sugeridas

1. Crear sitios web modernos con menús, hipertextos, enlaces a otros sitios
2. Explican el funcionamiento de un sitio web y de dónde proviene cada elemento del mismo y son capaces de elaborar elementos personalizados.

3.1 Qué es CSS

CSS (cuyas siglas significan "Cascading Style Sheets" [Hojas de estilo en cascada]) es un lenguaje que se usa para describir la apariencia y el formato de tu HTML.

Una hoja de estilo es un archivo que describe como debe verse un archivo HTML.

Las hojas de estilo en cascada se llaman así porque pueden usarse unas sobre otras para sumar reglas y aplicarlas todas a un mismo documento. Por ejemplo, si indicas que todos los párrafos deben tener la fuente de color azul, pero señalas que un párrafo específico debe tener fuente de color rojo, ¡CSS lo puede hacer!.

Ejemplo de código CSS:

Código CSS	Código HTML
<pre>p { color: red; } span { color: blue; }</pre>	<pre><!DOCTYPE html> <html> <head> <link type="text/css" rel= "stylesheet" href="stylesheet.css"/> <title>Fuentes atractivas</title> </head> <body> <p> ¡Soy un párrafo escrito en rojo pero una de mis palabras es azul!</p> </body> </html></pre>

En el ejemplo vemos una parte de código CSS que nos dice que los párrafos `<p>` van a estar en color rojo (`p { color: red }`) y que las etiquetas `<span>` serán de color azul (`span { color: blue }`). Cuando usamos estilos CSS, nos ahorramos escribir código HTML, ya que en pocas líneas de código CSS podemos definir que elementos repetitivos de la página como: fotos, párrafos, encabezados, etc. lleven el mismo estilo de fuente, color o tamaños, sin tener que escribir en cada elemento de la página los atributos de cada uno de ellos.

¿Por qué separar la forma de la función?

Existen dos razones principales para separar tu forma/formato (CSS) de tu contenido/estructura funcional (HTML):

Puedes aplicarle el mismo formato a diversos elementos de HTML sin tener que volver a escribir el código (p.ej.: `style="color:red"`) una y otra vez.

Puedes aplicarle una apariencia y un formato parecido a varias páginas de HTML desde un solo archivo CSS.

Ejercicio practico

Objetivo enlazar un estilo CSS a una pagina html

1. Abrimos Visual Studio Code y creamos un documento HTML



2. Escribimos el siguiente código:

```
<!DOCTYPE html>
<html>
<head>
  <link type="text/css" rel="stylesheet" href="stylesheet.css" />
  <title>Fuentes mucho más bonitas</title>
</head>
<body>
  <p> Mucho de esto es texto común pero una parte es <span>atractiva</span>!
    Podemos usar la fuente <span>que hicimos másbonita</span> para darle
    <span>estilo</span> a nuestro sitio web. Sería un <span>lío</span>
    hacer cada una de estas etiquetas <span>bonitas</span> una por una,
    pero es una papa con <span>CSS</span>!
  </p>
</body>
</html>
```

3. Guarda el archivo en una carpeta a la cual nombraras ejercicio 5, y guarda el archivo como ejercicio estilos css.html, abre el documento HTML en el navegador.
4. Ahora vamos a crear un nuevo documento (shortcut Control + N) y esta vez no escogeremos HTML, escogeremos HOJA DE ESTILOS y damos clic en abrir.
5. Escribe el siguiente código en el nuevo documento que has creado.

```
span {
  color: red;
  font-family: cursive;
}
```

6. Guarda el archivo con el nombre stylesheet.css, es importante que lleve ese nombre y que lo guardes en la misma carpeta donde has guardado el código html que escribimos anteriormente.
7. Ahora refresca la pantalla del navegador observa que ha cambiado.

Has notado que todas las palabras bajo la etiqueta cambiaron de color y se pusieron en itálica. Prueba hacer otras modificaciones de color y de fuente dentro del archivo Stylesheets.css guarda el archivo y refresca el navegador con tu página web abierta.

1.1 Mejor fuera de línea que en línea

Antes te mostramos cómo dar estilo inline (en línea) con HTML, así:

```
<p style="color:red">Red font!</p>
```

Esta no es la mejor forma de darle estilo a tu sitio web por las razones que acabamos de mencionar: tienes que escribir el mismo código una y otra vez, y si querés hacer un cambio de estilo importante en varios elementos tienes que cambiar cada una de las etiquetas de estilo. Con un único archivo CSS haces el cambio en un solo lugar. Existen dos formas de ubicar las CSS en un solo lugar. La primera es colocar tu CSS entre etiquetas, justo en el mismo archivo de código HTML. Estas etiquetas van dentro de las etiquetas de encabezado de tu página web; revisá el ejemplo:

```
<!DOCTYPE html>
<html>
  <head>
    <style>
      p {
        color: purple;
      }
    </style>
    <title>Resultado</title>
  </head>
  <body>
    <p>¡Mirá! ¡Soy violeta!</p>
  </body>
</html>
```

1.2 Enlazar estilos CSS

Ya sabes que deberías escribir tu CSS en un archivo aparte. Pero, ¿cómo te asegurás de que tu archivo HTML pueda ver esa información en CSS?

Lo puedes hacer agregando una etiqueta de enlace <link> (tal como viste en el primer ejercicio 5) entre las etiquetas de encabezado <head>...</head> de tu página HTML. Tu etiqueta de enlace <link> necesita tres atributos:

1. Un atributo type que siempre debe ser igual a "text/css"
2. Un atributo rel que siempre debe ser igual a "stylesheet"
3. Un atributo href que debe apuntar a la dirección web de tu archivo CSS (cuando trabajas offline guarda en la misma carpeta el archivo CSS)

Ejemplo: este fue el link que utilizamos para enlazar el código CSS con la página web del ejercicio 5 notarás que incluye los tres atributos.

```
<link type="text/css" rel="stylesheet" href="stylesheet.css" />
```

1.3 Etiquetas autocerrables

Esto nos hace acordar de algo que no podemos dejar de mencionar en HTML: las etiquetas autocerrables. Como nunca se agrega nada entre las etiquetas de enlace, está bien usar un solo par de `< >` para que sirvan como etiquetas de inicio y de cierre. Esto lo hacés de esta forma:

```
<link type="text/css" rel="stylesheet" href="dirección de archivo CSS"/>
```

Tal vez te hayas dado cuenta de que hacemos algo parecido con la etiqueta de imagen `<img>`:

```

```

La mayoría de las etiquetas no se autocierran, pero te vamos a mostrar cuáles sí, para ahorrarte tiempo y esfuerzo.

1.4 Sintaxis

La sintaxis de CSS es distinta de la de HTML a la que estás acostumbrado, pero no te preocupes: ¡es fácil de aprender! El formato general se ve así:

```
selector {  
    propiedad: valor;  
}
```

Un **selector** puede ser cualquier elemento de HTML, como `<p>`, `<img>`, o `<table>`. ¡Simplemente no lleva los `< >`!. Por ejemplo para hacer que el texto de un párrafo tenga color rojo con CSS, se escribe:

```
p {  
    color: red;  
}
```

Una **propiedad** es un aspecto de un **selector**. Por ejemplo, puedes cambiar el tipo de fuente (`font-family`), el color (`color`) y el tamaño de fuente (`font-size`) del texto en tus páginas web (además de otras cosas).

Un **valor** es una posible configuración de una propiedad. El color puede ser rojo, azul, negro, o casi cualquier color; el tipo de fuente (`font-family`) puede ser un montón de fuentes diferentes, y así sucesivamente.

Debés finalizar cada propiedad-valor con un punto y coma (;) para que CSS sepa que ya terminaste de trabajar con un par y que estás listo para trabajar con el siguiente.

1.5 Propiedades del selector

Otra ventaja genial de CSS es que puedes hacer que un selector tenga muchas propiedades. Por ejemplo, si quieres establecer la fuente, el color de fuente y el tamaño de fuente de un párrafo, solo tienes que escribir:

Código	Descripción
<pre>p { font-family: Arial; color: blue; font-size: 24px; }</pre>	Tres propiedades agrupadas en un solo selector, en este ejemplo hace que la fuente sea Arial color azul y 24px de tamaño

Recuerda: ¡termina cada par propiedad-valor con un punto y coma!

Ejercicio Práctico

Combinar selectores y propiedades

1. Escribe el siguiente código como html

```
<!DOCTYPE html>  
<html>  
  <head>  
    <link type="text/css" rel="stylesheet" href="stylesheet.css"/>  
    <title>Escribe un titulo</title>  
  </head>  
  <body>  
    <div>  
      <h3>¿Para qué sirve CSS?</h3>  
      <p>CSS sirve para darle estilo a las páginas de HTML</p>  
      <h3>¿Por qué hay que usarlo?</h3>  
      <p>Porque hace que las páginas web queden <span>realmente  
atractivas</span>. </p>  
      <h3>¿Que qué me parece?</h3>  
      <p>¡Me parece muy bueno!</p>  
    </div>  
  </body>  
</html>
```

2. Intenta que el color de todos los encabezados h3 sea red (rojo).
3. Haz que el tipo de letra (font-family) sea Courier en todos los párrafos. (Asegurate de escribir "Courier" con mayúscula inicial.)
4. El segundo párrafo tiene texto en medio de las etiquetas . Haz que el color de fondo (background-color) de esa sea amarillo (yellow).

NOTA: Escribe el código CSS y recuerda guardarlo con el nombre `stylesheets.css`, si cambias el nombre del archivo también cambia el nombre que coloques en la etiqueta `link` por el nombre con que guardaste la hoja de estilos, recuerda guardar ambos archivos en una misma carpeta.

Ejemplo:

```
selector {  
    propiedad: valor;  
}
```

Resumen

1. A medida que empiezas a agregar más y más pares **propiedad-valor** para cada selector CSS es importante que te recuerdes de poner punto y coma (;) al final de cada línea.
2. El punto y coma le indica a las CSS que es el fin de un par propiedad-valor y que es el momento de continuar con el siguiente. Si no pones punto y coma, se confundirá todo y tu página no se verá correctamente.
3. Además, no te olvides de que todos los pares propiedad-valor para un selector están en medio de llaves ({}).

Ejercicios: Encuentra los errores en el código y corrígelos.

Código con error	Escribe el código correcto
<pre>h3 { font-family: Verdana; color: blue; }</pre>	
<pre>p { font-family: Garamond; font-size: 16px; }</pre>	
<pre>h3 { font-family: Verdana color: blue }</pre>	
<pre>. { font-family: Garamond; font-size: 16px; }</pre>	
<pre>h3 fontfamily: Verdana; color blue }</pre>	
<pre>p { font-family: garamond; font-size: 16px; }</pre>	

Si bien es importante que tu sintaxis sea correcta, también es una buena idea escribir comentarios a medida que avanzas. Los buenos comentarios te van a ayudar a recordar de por qué hiciste algo de cierta manera (o van a ayudar a otra persona, si está leyendo tu código y no estás ahí para explicarlo).

Como ya viste, los comentarios de HTML se ven así:

```
<!-- ¡Soy un comentario! -->
```

Pero, los comentarios de CSS se ven así:

```
/* ¡Soy un comentario! */
```

La computadora ignora los comentarios cuando intenta resolver lo que deben hacer tus códigos HTML y CSS, pero escribir buenos comentarios es un buen hábito que deberías adquirir.

Ejercicio Práctico

1. Crea un documento HTML nombralo Ejercicio 6.html, crea una nueva carpeta donde guardarlo.
2. Escribe este código

```
<!DOCTYPE html>
<html>
<head>
  <!--Coloca aquí la etiqueta de tu enlace-->
  <title>Resultado</title>
</head>
<body>
  <h1>Cambiar por Verdana.</h1>
  <h3>Cambiar por Courier.</h3>
  <p>para a color violeta.</p>
</body>
</html>
```

3. Crea un documento CSS nuevo con el nombre stylesheet.css
4. Agrega una etiqueta <link> a stylesheet.css en medio de tus etiquetas <head></head>.
5. Usa Código CSS para lo siguiente
 - Cambia la fuente (font-family) del encabezado <h1> por Verdana. (Revisa de que la primera letra de Verdana esté en mayúscula). Usando código CSS
 - Cambia la fuente (font-family) del encabezado <h3> por Courier.
 - Haz que el color del texto del párrafo sea purple ("violeta" en inglés).
 - ¡Agrega un comentario (/* CSS comment */) a la pestaña CSS! Podés escribir lo que quieras. (No olvidar que los comentarios de HTML y los comentarios de CSS se escriben de forma diferente).

2. Medidas

Cuando te pedimos que ajustaras el tamaño de la fuente, especificamos que la unidad que tenías que usar era px (por "píxeles"), así:

```
p {  
  font-size: 10px;  
}
```

Un píxel es un punto en la pantalla de tu computadora. Especificar los tamaños de la fuente en píxeles es genial cuando quieres que el usuario vea en su pantalla exactamente lo que diseñaste en la tuya, pero se supone que ambas pantallas tienen el mismo tamaño.

Pero, ¿si el tamaño de la pantalla del usuario es muy diferente del tuyo (como la pantalla de un smartphone)? Aquí aparecen los **em**. (No hay que confundirlos con las etiquetas `<em>` `</em>` que usamos para énfasis).

La unidad de tamaño de fuente `font-size em` es una medida relativa: un **em** es igual al tamaño de fuente predeterminado en cualquier pantalla que el usuario esté usando. Esto los convierte en algo genial para las pantallas de los smartphones, ya que no tratan de decirle al smartphone exactamente cuál debe ser el tamaño de la fuente: sencillamente le indican, "Mirá: **1em** es el tamaño de fuente que usás casi siempre, así que **2em** es el doble de grande y **0,5em** es la mitad de ese tamaño".

Ten en cuenta que incluimos tres párrafos diferentes con `font-size 1em`, `0,5em`, y `2em`. Por ahora, puedes usar la unidad con la que te sientas más cómodo (**px** o **em**); nosotros solo queríamos que conocieras los **em** ahora para más adelante cuando los ocupes ya sepas de que hablamos.

También te pedimos que cambiaras el tipo de letra (`font-family`) de ciertos elementos usando CSS. Nos viste usar las fuentes Verdana, Courier y Garamond.

¿Pero cuántas fuentes conoce CSS?

La respuesta es: depende. La mayoría de las computadoras van a entender las fuentes más comunes, como Verdana, Courier y Garamond, pero cada computadora tiene instaladas diferentes fuentes. Lo bueno es que CSS tiene algunas fuentes predeterminadas incorporadas para que tus usuarios vean lo que vos querés. Son:

1. **serif**: una fuente con pequeños remates decorativos en los finales de los trazos que componen las letras. Busca "serif" para ver a qué nos referimos.
2. **sans-serif**: una fuente sin adornos, como esta. No tiene las cositas de los extremos de las letras que tiene la fuente serif.
3. **cursive**: es una fuente que parece hecha a mano alzada. Se ve como escritura cursiva.

Ejemplos:

```
h1 {
```

```
font-family: serif;
```

```
}
```

```
h2 {
```

```
font-family: sans-serif;
```

```
}
```

```
h3 {
```

```
font-family: cursive;
```

```
}
```

No tienes que usar obligatoriamente un valor predeterminado como `cursive` o `sans-serif`: puedes indicarle a CSS que pruebe varias fuentes, yendo de una a la siguiente si la que estás buscando no está disponible.

Por ejemplo, si escribes:

```
p {
```

```
font-family: Tahoma, Verdana, sans-serif;
```

```
}
```

CSS intentará primero aplicar Tahoma a tus párrafos. Si la PC del usuario no tiene esa fuente, a continuación probará con Verdana, y si esa no funciona, mostrará una fuente `sans-serif` predeterminada.

3. Color de fondo, altura y ancho

¿Recuerdas a la etiqueta `<div>` y cómo lo usamos para hacer esos bloques multicolores?

Hay tres propiedades para las cuales deberás determinar valores:

- Color de fondo (`background-color`), que se determina con un color o valor hexa
- Altura (`height`), que se determina con un valor en píxeles
- Ancho (`width`), que también se mide en píxeles

La mejor manera de aprender es practicar realiza el siguiente ejercicio.

Ejercicio práctico

1. Crea un documento HTML nombralo Ejercicio 7.html, crea una nueva carpeta donde guardarlo.
2. Escribe este código

```
<!DOCTYPE html>
<html>
  <head>
    <link type="text/css" rel="stylesheet"
href="stylesheet.css"/>
    <title>Resultado</title>
  </head>
  <body>
    <div></div>
  </body>
</html>
```

3. Crea un documento CSS nuevo con el nombre stylesheet.css
4. Haz que background-color tenga como valor #cc0000,
así: background-color: #cc0000;
5. Haz que height tenga como valor 100px,
así: height: 100px;
6. Haz que width también tenga como valor 100px.
así: width: 100px;

Nota: La clave esta en usar la etiqueta <div>.

4. Bordes

Muchos elementos de HTML admiten la propiedad borde (border). Esta puede ser especialmente útil con las tablas. La propiedad border a su vez admite varios valores. Por ejemplo, para hacer un borde de 2 píxeles de ancho, sólido, y de color rojo, tienes que escribir por ejemplo:

```
selector {
  border: 2px solid red;
}
```

Repasemos: El uso de tablas

1. Crea un documento HTML nombralo Ejercicio 8.html, crea una nueva carpeta donde guardarlo.
2. Escribe este código

```
<!DOCTYPE html>
<html>
  <head>
    <link type="text/css" rel="stylesheet"
href="stylesheet.css"/>
    <title></title>
  </head>
  <body>
    <table>
      <thead>
        <th colspan="3">Nueve bloques</th>
      </thead>
      <tbody>
        <tr>
          <td></td>
          <td></td>
          <td></td>
        </tr>
        <tr>
          <td></td>
          <td></td>
          <td></td>
        </tr>
        <tr>
          <td></td>
          <td></td>
          <td></td>
        </tr>
      </tbody>
    </table>
  </body>
</html>
```

3. Crea un documento CSS nuevo con el nombre stylesheet.css
4. En código CSS haz lo siguiente:
 - Haz que tus selectores td (celdas de datos de tabla) tengan una altura de 50 píxeles (height de 50px), de manera que podamos verlos mejor cuando agreguemos nuestro borde.
5. Dale a tus td una propiedad border de 1px de color azul punteado (**dashed blue**).
6. Dale a tu tabla (table) una propiedad border de 1px de color negro liso (**solid black**).

Ejemplo:

```
td{
    height:50px; border: 1px dashed blue;
}
table{
    border:1px solid black;
}
th{
    color:red;
}
```

5. Enlaces y "text-decoration" (decoración de texto)

Los enlaces tienen muchas de las propiedades que tiene el texto normal: puedes cambiarles la fuente, el color, el tamaño, etc. Pero los enlaces también tienen la propiedad `text-decoration`, que se puede modificar para darles un toque un poco más personalizado. Probablemente estés acostumbrado a ver enlaces que son de color azul y están subrayados, ¿no? Bueno, no tiene que ser sí o sí de esa manera.

Para cambiar la decoración haces lo siguiente:

Ejemplo: Código en HTML	En CSS:
<pre><!DOCTYPE html> <html> <head> <link type="text/css" rel="stylesheet" href="stylesheet.css"/> <title>Resultado</title> </head> <body> <p> ¡El siguiente enlace te lleva a Google! </p> Google </body> </html></pre>	<pre>a { color: #cc0000; text-decoration: none }</pre>

Escribe ambos códigos para que vea como funciona estas decoraciones: `overline`, `line-through`, `underline`.

6. Crear botones

Vamos a empezar. Lo primero es lo primero: necesitamos crear nuestro botón. Esto lo hacemos con las etiquetas <div></div>.

Como crear un boton que redireccione a facebook por ejemplo:

Este seria el código html

```
HTML
<!DOCTYPE html>
<html>
  <head>
    <link type="text/css" rel="stylesheet" href="stylesheet.css"/>
    <title>Titulo</title>
  </head>
  <body>
    
    <p>Esta es mi pagina de facebook.</p><br/><br/>
    <div>

    </div>
  </body>
</html>
```

Y asi el complemento en CSS

CSS

```
img {
  display: block;
  height: 100px;
  width: 300px;
  margin: auto;
}

p {
  text-align: center;
  font-family: Garamond, serif;
  font-size: 18px;
}

div {
  height: 50px;
  width: 120px;
  border-color: #6495ED;
  background-color: #BCD2EE;
  border-style: solid
}
```

Dar forma al botón

Este ejercicio incluye una nueva propiedad llamada `border-radius` (radio del borde). Esta propiedad modifica las esquinas de los objetos de HTML; es así como se consigue hacer esos geniales botones redondos.

Ejemplo:

`border-radius: 5px;`

Posición del botón

Así es como funciona:

1. `margin: auto;` es el estilo que usás para decirle al navegador: "Asegurate de dejar márgenes iguales a ambos lados de este elemento de HTML". Cuando los márgenes son iguales, el objeto queda centrado.
2. `text-align: center;` se usa para centrar elementos de texto.

Este sería el código en CSS del boton terminado

```
img {
    display: block;
    height: 100px;
    width: 300px;
    margin: auto;
}

p {
    text-align: center;
    font-family: Garamond, serif;
    font-size: 18px;
}

div {
    height: 50px;
    width: 120px;
    border-color: #6495ED;
    background-color: #BCD2EE;
    margin: auto;
    text-align: center;
    border-radius: 5px;
    border-style: solid
}

div {
    font-family: Verdana;
    color: blue;
}

a {
    color: blue;
    text-decoration: none
}
```

Escribe este código modificalo a tu gusto prueba con distintas combinaciones de colores y letras así como radios del borde, puedes conseguir bordes más redondos o rectos al modificar el valor de `border-radius`.

6. Elementos HTML como selectores

Ya usamos varios elementos de HTML como selectores de CSS: le dimos estilo a las etiquetas de HTML `<h1></h1>` con el selector de CSS `h1`, a `<p></p>` con `p`, y así sucesivamente.

Quizás ya adivinaste esta parte, pero si no, vamos a decirla de una vez: ¡cualquier elemento de HTML puede ser un selector de CSS! Podés modificar las `<ul>`, `<table>`, e incluso la etiqueta `<body>` completa, seleccionando `ul`, `table`, y `body`, respectivamente.

Ejemplo: Para cambiar el color del cuerpo de la pagina web:

```
body {  
    background-color: black;  
}
```


6.1 Selectores múltiples

Como ya hemos visto, es posible anidar elementos HTML dentro de otros, así:

```
<div>
```

¿Y si quieres enfocarte en las <p> que están dentro de dos <div>, y no en todas las <p>?

Puedes seleccionarlas en la pestaña CSS de esta forma:

```
div div p {  
    /* ¡Lo de CSS! */  
}
```

HTLM	CSS
<pre><!DOCTYPE html> <html> <head> <link type="text/css" rel="stylesheet" href="stylesheet.css"/> <title>Result</title> </head> <body> <h3>Soy una simple letra antigua</h3> <div> <h3>Y yo también</h3> <div> <h3>¿y yo?</h3> <div> <h3>¡Estoy a punto de ser roja!</h3> </div> </div> </div> </body> </html></pre>	<pre>div div div h3 { color: red; }</pre>
<i>Intenta escribir ambos código para que veas su resultado.</i>	

6.2 Un selector para controlar los demas selectores

También existe un selector muy especial que puedes usar para aplicar el estilo de CSS a cada elemento de la página: el selector *. Por ejemplo, si escribes:

CSS

```
* {  
  
    border: 2px solid black;  
  
}
```

Vas a crear un borde continuo de dos píxeles de ancho alrededor de cada elemento en la página de HTML.

Ejercicio 1:

Usá el selector universal para colocar un borde azul punteado de valor 1px dashed blue alrededor de cada elemento de la página.

Usa este código HTML

```
<!DOCTYPE html>
<html>
  <head>
    <link type="text/css" rel="stylesheet"
href="stylesheet.css"/>
    <title>Resultado</title>
  </head>
  <body>
    <h3>Cajas dentro de cajas</h3>
    <div>
      <p>Párrafo uno</p>
    </div>
    <div>
      <p>Párrafo dos</p>
    </div>
    <div>
      <p>Párrafo tres</p>
    </div>
  </body>
```

Escribe el código CSS.

Ejercicio 2

1. Escribe este código HTML en visual studio code.

```
<!DOCTYPE html>
<html>
  <head>
    <link type="text/css" rel="stylesheet" href="stylesheet.css"/>
    <title>Mostrá lo que sabés hacer</title>
  </head>
  <body>
    <p>Estoy a punto de tener un adorable color verde azulado.</p>
    <p>¡Yo también!</p>
    <p>Yo creo que voy a hacer lo mismo.</p>
    <div>
      <p>¡Vamos a tener un color escarlata muy llamativo!</p>
      <p>Estaba pensando más en un color bermellón.</p>
      <p>¡No, color carmesí!</p>
    </div>
  </body>
</html>
```

2. Guarda el ejercicio.

3. Crea el archivo Stylesheet.css

4. En el archivo CSS haz lo siguiente:

- Que todos los textos de párrafo tengan el color hexadecimal #00E5EE.
- Todos los textos de párrafo para los párrafos anidados dentro de las etiquetas div tengan el color hex #CC0000. (¿De qué color serán si en el paso 1 ya se les cambió el color a #00E5EE?
- Agrega un borde con el color #3A5FCD alrededor de cada elemento HTML. Puede ser línea continua, punteada, discontinua, 2px, 3px, lo que quieras.

Ayudate de estos consejos:

Al texto del párrafo que está dentro de las etiquetas div se le da un tratamiento con el selector p de esta forma:

```
p {
    /* aqui colocas tus estilos */
}
```

A todo el texto del párrafo que está dentro de las etiquetas div se le da un tratamiento con el selector div p de esta forma:

```
div p {
    /* aqui colocas tus estilos */
}
```

A cada elemento de HTML se le da un tratamiento con el selector universal de esta forma:

```
* {  
  
    /* aqui colocas tus estilos */  
  
}
```

Ya que decir "las <p> dentro de las <div>" es más específico que decir "todas las <p>", los párrafos fuera de los div tendrán color verde azulado y aquellos dentro de los div tendrán color rojo oscuro. Este comportamiento se llama cascada, y vamos a hablar de eso en la siguiente sección.

7. Ramificación

Imaginá que un documento de HTML es como un árbol: los elementos se "ramifican" desde el tronco principal (las etiquetas <html></html>). Las dos primeras ramas más grandes son <head> y <body>, y las ramas se multiplican y se hacen más delgadas a medida que agregas elementos como las <div>, <title>, y el texto (encabezados y párrafos). Si imaginás que la etiqueta <html> es el tronco del árbol, puedes imaginarte que sus ramas principales <head> y <body> son sus hijos. Ambas etiquetas son hijos de <html>, y <html> es su elemento padre. Como ambos son hijos directos de <html> (es decir que tienen solamente un elemento de distancia), son hermanos. Como en una familia de verdad, los elementos tienen hijos, nietos, bisnietos, y así sucesivamente (aunque nosotros no hacemos esta distinción con HTML; un hijo de un elemento, y todos los hijos de ese hijo, son hijos del primer padre).

Ejemplo:

```
<!DOCTYPE html>  
<html> <!-- El tronco del árbol -->  
    <head> <!-- Hijo de html, pariente del titulo,  
        hermano del cuerpo -->  
        <title></title> <!-- Hijo directo del encabezado,  
            hijo del encabezado Y de html -->  
    </head>  
    <body> <!-- Hijo de html, pariente de p,  
        hermano del encabezado -->  
        <p></p> <!-- Hijo directo del cuerpo,  
            hijo del cuerpo Y de html -->  
    </body>  
</html>
```

Nota: un elemento es hijo de CADA elemento que lo encierra, ¡incluso si ese elemento está a algunas "ramas" de distancia. Ahora que ya tienes una idea de cómo están estructurados los documentos HTML, es hora de ver si has entendido como saltar de rama en rama.

Ejercicio:

No olvides, puedes referirte a un elemento que es hijo de otro elemento así:

```
div div p { /* Código de CSS */ }
```

En este caso, vamos a tomar cualquier `<p>` que esté anidada en algún lugar dentro de una `<div>` que está anidada en algún lugar dentro de otra `<div>`. Si quieres tomar hijos directos (es decir, los elementos que están directamente anidados dentro de otro elemento, sin elementos intermedios) puedes usar el símbolo `>`, así:

```
div > p { /* Código de CSS */ }
```

Esto se refiere únicamente en las `<p>` que estén anidadas directamente dentro de las `<div>`; no tomará los párrafos que estén anidados dentro de listas que, a su vez, estén anidadas dentro de las `<div>`.

Usaremos este código HTML para practicar:

```
<html>
  <head>
    <link type="text/css" rel="stylesheet" href="stylesheet.css"/>
    <title>Máximo desafío de texto</title>
  </head>
  <body>
    <p>Introducción: Efecto cascada con CSS</p>
    <div>
      <p>Sinopsis: Cuando estableces una propiedad de un selector como 'p'
      con un cierto valor, ese valor se aplica a todas las etiquetas
      p. Sin embargo, si cambiás esa misma propiedad a un valor diferente
      para una instancia de p más específica, ese cambio sobreescribirá
      la 'regla general'.
      </p>
      <ul>
        <li><p>Si dices: p { font-family: Garamond}, todas las 'p' tendrán
        fuente Garamond.
        </p></li>
        <li><p>PERO si dices li p {font-family: Verdana}, las 'p' fuera de las
        'li' tendrán fuente Garamond, y las 'p' DENTRO de las 'li'tendrán fuente
        Verdana.
        </p></li>
        <li><p>Cuanto más específicos sean tus selectores, más importancia
        le da CSS al estilo que apliques.</p></li>
      </ul>
    </div>
    <p>Resumen: Una mayor especificidad hace que CSS le dé prioridad a ese
    estilo particular.</p>
  </body>
</html>
```

Lo que debes hacer desde CSS

1. Haz que todas las etiquetas <p> tengan propiedad font-family con valor Garamond. (¡NO uses el selector universal para hacerlo!)
2. Haz que el párrafo de introducción y el de resumen tengan la propiedad font-weight con valor bold (esta propiedad es nueva, pero funciona igual que las otras que has aprendido).
3. Haz que los párrafos en la lista no ordenada tengan la propiedad decoración de texto con valor underline.

Buena suerte

Así sería el aspecto final:

Introducción: Efecto cascada con CSS

Sinopsis: Cuando establecés una propiedad de un selector como 'p' con un cierto valor, ese valor se aplica a *todas* las etiquetas p. Sin embargo, si cambiás esa misma propiedad a un valor diferente para una instancia de p más específica, ese cambio *sobrecribirá* la 'regla general'.

- Si decís: `p { font-family: Garamond }`, todas las 'p' tendrán fuente Garamond.
- PERO si decís `li p { font-family: Verdana }`, las 'p' fuera de las 'li' tendrán fuente Garamond, y las 'p' DENTRO de las 'li' tendrán fuente Verdana.

- Cuanto más específicos sean tus selectores, más importancia le da CSS al estilo que apliques.

Resumen: Una mayor especificidad hace que CSS le dé prioridad a ese estilo particular.

7.1 Clases

Como hemos visto, ciertos selectores "sobrecribirán" otros si tienen un **valor de especificidad mayor**. `ul li p { }` es CSS más específico que simplemente `p { }`, así que, cuando CSS ve etiquetas que son <p> y están dentro de listas no ordenadas, aplicará el estilo más específico (`ul li p { }`) al texto dentro de las listas.

Existen dos selectores que son aún más específicos que los selectores anidados como los que aparecen en el ejemplo:

Selectores de clase (class) y de identificadores (id).

```
<p id="intro">Efecto cascada con CSS</p>
```

```
<li><p class="list_item">PERO si dices li p {font-family: Verdana}
```

Los elementos de HTML pueden ser selectores de CSS, pero como vimos con el selector universal *, no son los únicos selectores disponibles. Existen dos selectores importantes que puedes usar, además del selector universal y de los elementos de HTML: los selectores de clase y de identificadores.

Ejemplo:

```
.red {  
  color: red;  
}  
#rogue {  
  color: #FF00FF;  
  font-weight: bold;  
  font-family: cursive;  
}
```

Las clases son útiles cuando tienes un montón de elementos que deberían tener el mismo estilo. En vez de aplicar las mismas reglas a varios selectores, simplemente puedes aplicar la misma clase a todos esos elementos de HTML y luego definir el estilo para esa clase en la pestaña de CSS. Las clases son asignadas a los elementos de HTML usando la palabra `class` y un signo de igual (=), así:

```
<div class="cuadrado"></div>
<img class="cuadrado"/>
<td class="cuadrado"></td>
```

En CSS, las clases se identifican con un punto (.), así:

```
.cuadrado {
  height: 100px;
  width: 100px;
}
```

Esto te permite tomar elementos de diferentes tipos y darles el mismo estilo.

Ejercicio práctico

1. Escribe este código HTML

```
<!DOCTYPE html>
<html>
  <head>
    <link type="text/css" rel="stylesheet" href="stylesheet.css"/>
    <title>Resultado</title>
  </head>
  <body>
    <!--Agrega tus elementos HTML con la clase "elegante" a continuación.-->
    `</body>`

</html>
```

2. Crea la clase elegante
3. Escribe el código CSS para la clase elegante
4. Agrega los estilos que quieras a la clase que creaste, puede ser color, fuente, tamaño, fondo, etc.

7.2 Identificadores

Por otra parte, los identificadores son geniales cuando tienes exactamente un elemento que tiene que tener cierto estilo. Los identificadores son asignados a los elementos de HTML usando la palabra `id` y un signo de igual (=):

```
<div id="primero"></div>
<div id="segundo"></div>
<p id="intro"></p>
```

Los identificadores se reconocen en CSS con un signo de numeral (#):

```
#primero {  
height: 50px;  
}  
#segundo {  
height: 100px;  
}  
#intro {  
color: #FF0000;  
}
```

Esto te permite aplicarle estilo a una sola instancia de un selector, en vez de a todas las instancias.

Ejercicio práctico

Utiliza este código html para el ejercicio.

```
<!DOCTYPE html>  
<html>  
  <head>  
    <link type="text/css" rel="stylesheet" href="stylesheet.css"/>  
    <title>Resultado</title>  
  </head>  
  <body>  
    <h2>Introducción</h2>  
    <h3>Clases e identificadores en CSS</h3>  
    <p>Las clases e identificadores en CSS son superfáciles. ¡En este momento  
los estás usando!</p>  
    <h3>Selectores regulares de HTML</h3>  
    <p>Si no usás una clase o un identificador, un elemento de HTML simplemente  
tendrá el estilo regular de CSS para ese elemento, o el estilo  
predeterminado, si no especificás ninguno en particular en la hoja de  
estilos. </p>  
  </body>  
</html>
```

2. Crea el archivo `stylesheet.css`
3. Escribe el código necesario para realizar lo siguiente:

En el código HTML hacer lo siguiente

- Dale al encabezado h2 el identificador "intro".
- Dale al primer encabezado h3 y al primer p una clase de "destacado". No les hagas nada ni al segundo h3 ni a p.

y en código CSS

- Establece el color del identificador #intro en #B83C3A. Establece el color de la clase .destacado en #F7AC5F y su propiedad font-family como Verdana.

Algo de ayuda:

HTLM

```
class= "nombredeclase"  
id="nombredeidentificador"
```

CSS

```
.nombre de clase { /* ¡Código de CSS! */ }  
#nombredeidentificador { /* ¡Código de CSS! */ }
```

7.4 Selectores de pseudoclase

Ya aprendiste sobre los selectores de clase. Ahora vamos a aprender sobre los selectores de **pseudoclase**. Un selector de **pseudoclase** es una forma de acceder a los elementos de HTML que no son parte del árbol del documento (¿recuerdas la estructura de árbol sobre la que hablamos anteriormente?).

Por ejemplo, es muy fácil ver en qué lugar del árbol va un enlace. Pero, ¿en qué lugar encontrarías la información que dice si se ha hecho clic o no sobre un enlace? ¡Eso no está ahí!

Los selectores de pseudoclase nos permiten darle un estilo a este tipo de cambios en nuestro documento de HTML. Por ejemplo, vimos que podemos cambiar la propiedad decoración de texto de un enlace para que no se vea siempre azul y subrayado. Usando los pseudo-selectores se controla la apariencia de enlaces que han sido y que no han sido visitados; e incluso de aquellos sobre los que el usuario pasa el cursor pero en los que aún no ha hecho clic.

La sintaxis de CSS para los pseudoselectores es:

```
selector:selector_de_pseudoclase {  
propiedad: valor;  
}
```

Simplemente es un signo adicional de dos puntos (:).

Otro ejemplo:

```
a:hover {  
color: #cc0000;  
font-weight: bold;  
text-decoration: none;  
}
```

7.5 Enlaces

Existe una cantidad de selectores de pseudoclases útiles para los enlaces, incluidos los siguientes:

1. `a:link`: Un enlace sin visitar.
2. `a:visited`: Un enlace que ha sido visitado.
3. `a:hover`: Un enlace sobre el que pasás el cursor.

Ejercicio

1. Usa el siguiente código HTML para el ejercicio

```
<!DOCTYPE html>
<html>
  <head>
    <link type="text/css" rel="stylesheet" href="stylesheet.css"/>
    <title></title>
  </head>
  <body>
    <div>
    </div>
  </body>
</html>
```

2. Instrucciones:

- Agrega tres enlaces en medio de las etiquetas div. ¡Pueden llevar a los sitios web que quieras!
- En el código CSS, establece la propiedad de todos los `a:link` para que no tengan decoración de texto y dales un color de `#008B45`.
- Establece el color de todos los `a:hover` en `#00FF00`.
- Establece el color de todos los `a:visited` en `#EE9A00`.

7.6 Pseudoclase `first-child` y `nth-child`

Otro selector de pseudoclase útil es *first-child*. Se usa para aplicar el estilo únicamente a los elementos que son los primeros hijos de sus padres.

Por ejemplo:

```
p:first-child {
color: red;
}
```

Esto hace que todos los párrafos que sean los primeros hijos de sus padres tengan color rojo. Se puede seleccionar cualquier hijo de un elemento después del primer hijo, con el selector de pseudoclase *nth-child*; simplemente tenés que agregar el número de hijo entre paréntesis después del selector de pseudoclase.

Por ejemplo:

```
p:nth-child(2) {
color: red;
}
```

Pondrá en color rojo a todos los párrafos que sean los segundos hijos de su elemento padre. El elemento que es el hijo va antes de `:nth-child`; su elemento padre es el elemento que lo contiene.

Ejemplo:

1. Escribe este código HTML

```
<html>
<body>
  <div>
    <p>¡Yo soy el primer hijo!</p>
    <p>Nosotros no.2</p>
    <p>Nosotros no.3</p>
    <p>Nosotros no.4</p>
    <p>Nosotros no.5</p>
    <p>Nosotros no.6</p>
    <p>Nosotros no.7</p>
  </div>
</body>
</html>
```

2. Ahora escribe el código CSS para conseguir los siguientes resultados
 - Establece la propiedad `font-family` del segundo párrafo como `Tahoma`.
 - Establece el color del tercer párrafo como `#CC0000`.
 - Establece el color de fondo del cuarto párrafo como `#00FF00`.
 - Establece la propiedad `font-size` del quinto párrafo como `22px`.

Aquí una ayuda:

```
1  p:first-child {
2  font-family:cursive;
3  }
4  p:nth-child(2) {
5  font-family:Tahoma;
6  }
```

Este número determina el párrafo que se está modificando

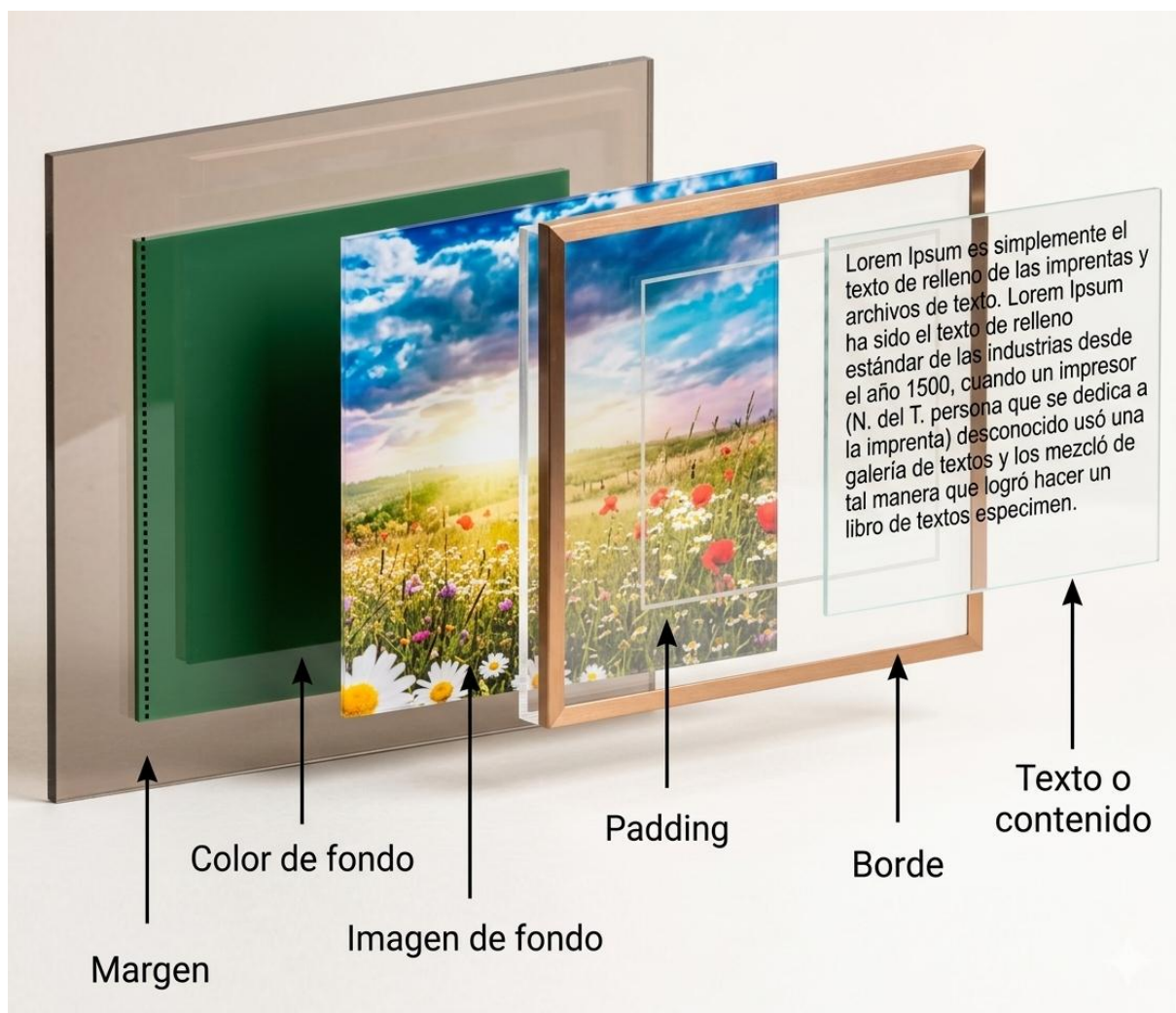
8. Posicionamiento

Ahora que sabes todo acerca de CSS, es hora de conocer la última (pero no por eso la menos importante) pieza del rompecabezas: **el posicionamiento**.

Si sabes controlar la posición de los elementos de HTML podrás tener mucho más control del aspecto de tus páginas. Tus `<div>` ya no estarán una encima de la otra. (Salvo que quieras que lo estén.)

Como lo mencionamos, los elementos están ubicados dentro de la página en lo que se conoce como modelo de caja de CSS. Cada elemento de HTML es como una pequeña caja o contenedor que encierra las imágenes y el texto que especifiques.

Observá la imagen que está abajo: así es como se ve el modelo de caja. (Veremos los detalles de los márgenes, los bordes y el relleno en la siguiente sección.)



Como viste, la caja más externa de cada elemento ocupaba el ancho de toda la página. Por eso, hasta ahora tus elementos de HTML han estado uno encima del otro: ocupan el ancho completo de la página de forma predeterminada.

Podemos cambiar todo esto con la primera propiedad de posicionamiento que vamos a aprender: la propiedad `display`. Vamos a aprender cuatro valores posibles.

1. **block (bloque):** esto convierte al elemento en una caja de bloque. ¡No permite que se coloque nada a su lado en la página! Ocupa todo el ancho.
2. **inline-block (bloque en línea):** Esto convierte al elemento en una caja de bloque, pero permite que otros elementos se coloquen al lado en la misma línea.

3. **inline (en línea):** Esto hace que un elemento se coloque en la misma línea que otro elemento, pero sin darle formato de bloque. Solamente ocupa el ancho que requiere (no ocupa la línea completa).
4. **none (ninguno):** Esto hace que el elemento y su contenido desaparezcan de la página por completo.

Como lo mencionamos antes, cualquier elemento que tenga características de bloque (por ejemplo, un párrafo) automáticamente ocupará el ancho completo de la página, sin importar si le asignás mucho o poco contenido.

Sin embargo, si especificamos una propiedad `display` de valor `inline-block`, nuestros bloques continúan siendo bloques, pero podemos ubicarlos uno al lado del otro en la misma línea.

Ejemplo

1. Escribe este código HTML

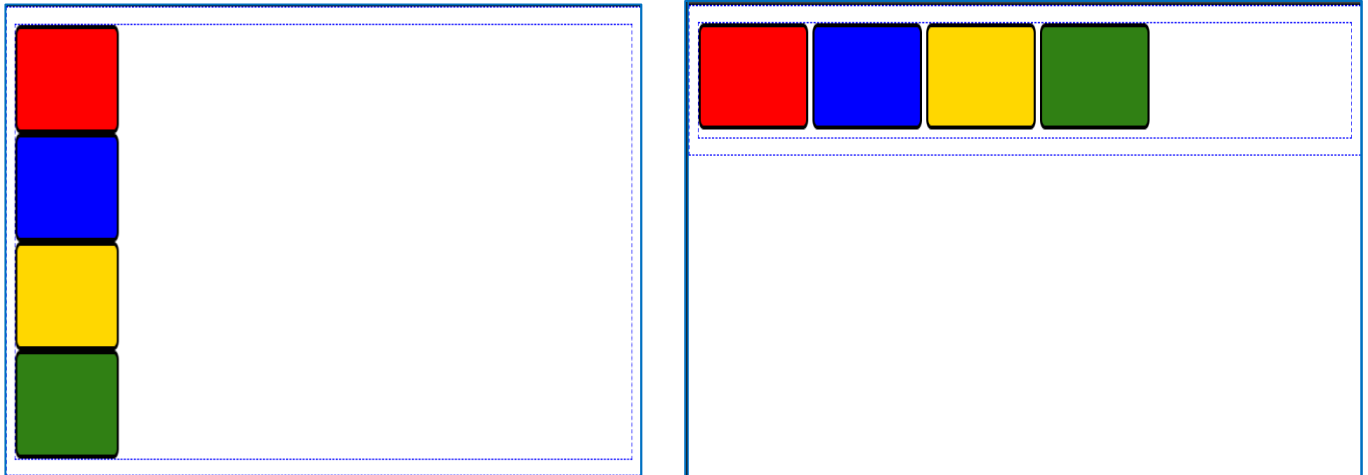
```
1  <!DOCTYPE html>
2  <html>
3      <head>
4          <link type="text/css" rel="stylesheet" href="stylesheet.css"/>
5          <title>Resultado</title>
6      </head>
7      <body>
8          <div id="uno"></div>
9          <div id="dos"></div>
10         <div id="tres"></div>
11         <div id="cuatro"></div>
12     </body>
13 </html>
```

2. Escribe este código CSS

```
1  * {
2      border: 1px dashed blue;
3  }
4
5  div {
6      height: 50px;
7      width: 100px;
8      border: 2px solid black;
9      border-radius: 5px;
10 }
11
12 #uno {
13     background-color: #FF0000;
14 }
15
16 #dos {
17     background-color: #0000FF;
18 }
19
20 #tres {
21     background-color: #FFD700;
22 }
23
24 #cuatro {
25     background-color: #308014;
26 }
```

3. Salva ambos archivos html y CSS y visualiza el resultado en el navegador
4. Modifica el código CSS agregando esta línea de código: **display:inline-block** en la sección de las etiquetas **div**, debajo de **border-radius:5px**
5. Compara los resultados

ESQUEMAS COMPONENTES DE COMPARACIÓN (RESULTADOS)

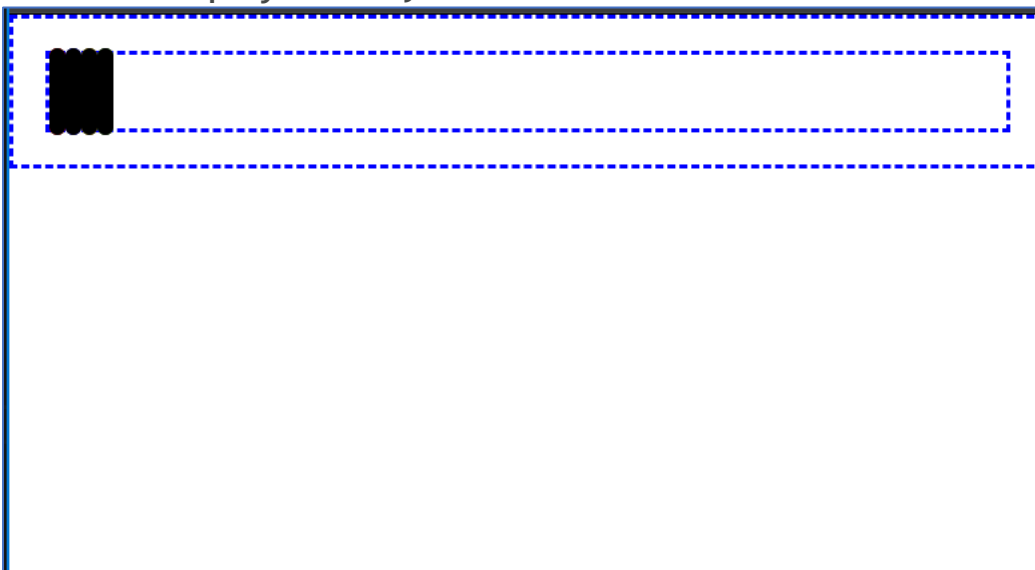


Al usar **display:inline-block**, cambiamos el orden de los bloques de vertical a horizontal

Todas tus `<div>` se ubicaron sobre la misma línea. Ya puedes comenzar a ver cómo este tipo de posicionamiento puede ser útil para las barras de navegación, como la que se encuentran en la parte superior de la páginas web de muchos sitios conocidos en internet.

El valor **inline-block** te permite ubicar varios elementos de bloque en la misma línea. El valor **inline** ubica todos tus elementos uno al lado del otro, pero no como bloques: los elementos no conservan sus dimensiones.

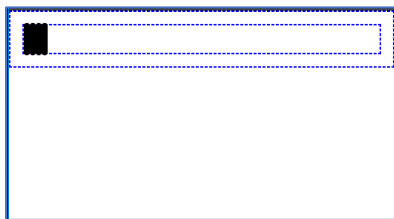
Haciendo uso del ejercicio anterior, reemplaza la línea de código **display:inline-block** y dejala solo como **display:inline**; observa el resultado.



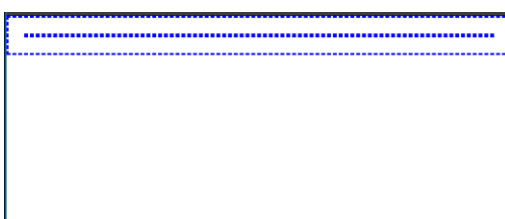
Lo bueno es que **inline** ubica todos tus elementos en una sola línea. Lo malo es que no conserva sus características de caja: como pudiste ver, todas las `<div>` quedaron reducidas al ancho más pequeño posible.

El valor de **display:inline** se ajusta mejor a los elementos de HTML que son bloques de forma predeterminada, como los encabezados y los párrafos.

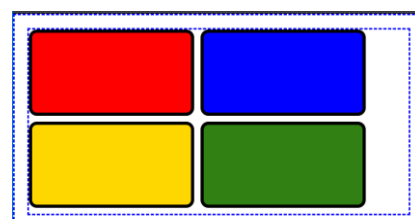
Por último, vamos a probar el valor de **display:none**. Como imaginarás, este valor evita que la página muestre el elemento seleccionado. Como no imaginarás, esto además elimina completamente el elemento de la página, incluyendo cualquier hijo o contenido que tenga. Observa los diferentes resultados resultados:



Display:inline



Display:none

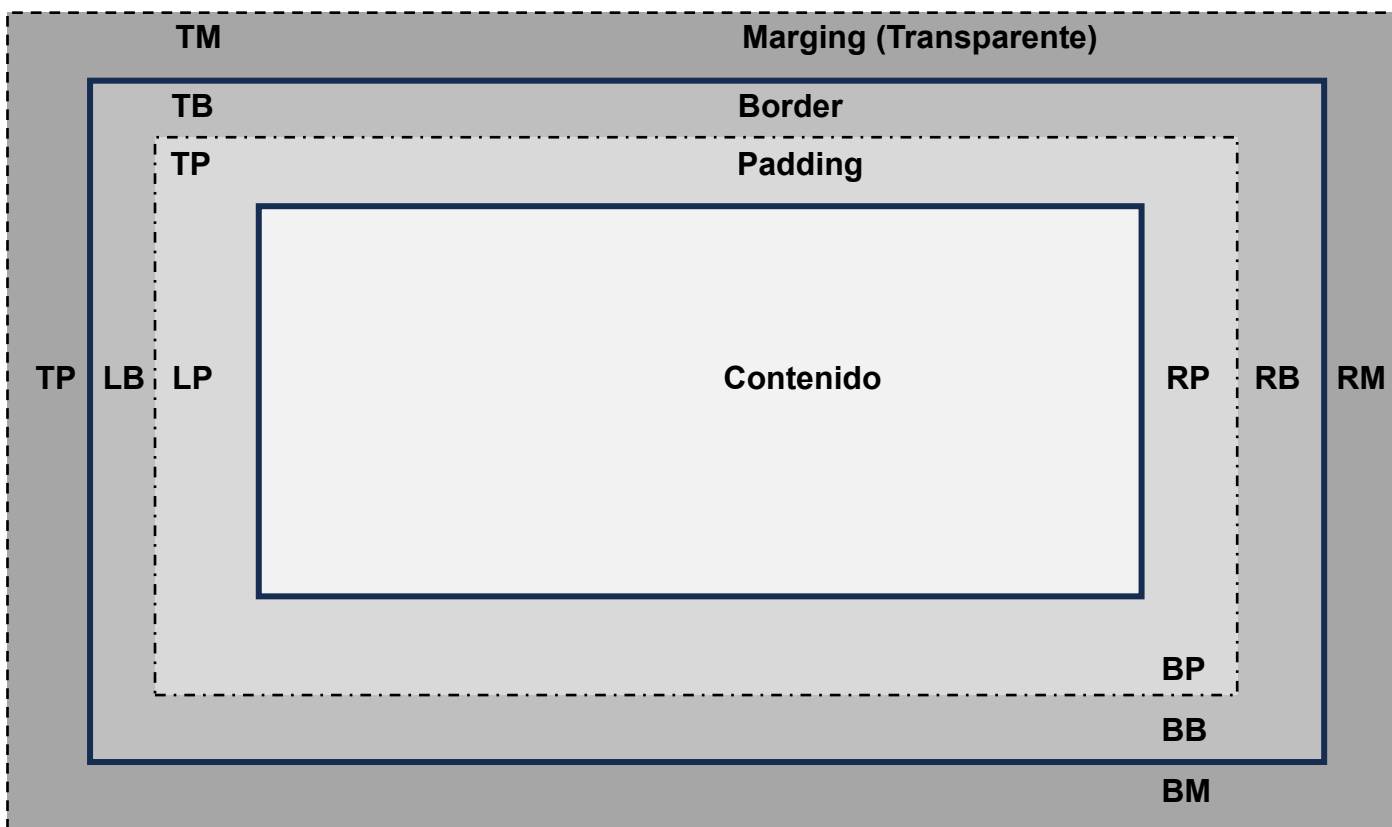


Display:inline-block

9. Layout o bosquejo

Ahora que conoces más la propiedad "**display**" y el modelo de caja, podemos ver en profundidad los detalles del comportamiento de cada caja.

Observa la siguiente imagen:



Abreviaturas de referencia:

- | | | | |
|--------------------|---------------------|----------------------|-----------------------|
| * TM = Top Margin | * LM = Left Margin | * RM = Right Margin | * BM = Bottom Margin |
| * TB = Top Border | * LB = Left Border | * RB = Right Border | * BB = Bottom Border |
| * TP = Top Padding | * LP = Left Padding | * RP = Right Padding | * BP = Bottom Padding |

1. **Margin** (margen) es el espacio que está alrededor del elemento. Cuanto más grande el margen, más espacio hay entre nuestro elemento y los que lo rodean. Podemos ajustar el margen para mover nuestros elementos de HTML para que estén más cerca o más lejos unos de otros.
2. **Border** es el borde del elemento. Es lo que estuvimos haciendo visible cada vez que establecimos la propiedad border.
3. **Padding** (relleno) es el espacio entre el contenido y el borde. Con CSS, podemos ajustar este valor para mover el borde más cerca o más lejos del contenido.
4. **Content** es el contenido que está dentro de la caja. Si estamos hablando de un elemento, el contenido es el texto del párrafo.

En el diagrama vas a ver abreviaturas como TM, TB, y TP. Estas quieren decir "top margin" (margen superior), "top border" (borde superior), y "top padding" (relleno superior). Como veremos, podemos ajustar los rellenos, bordes y márgenes superiores, derechos, izquierdos e inferiores en forma individual.

9.1 Márgenes

Vamos a empezar con los márgenes. Si ajustamos los márgenes no solo desplazamos nuestro elemento con relación a otros elementos de la página, sino que también lo hacemos en relación con las "paredes" del documento de HTML.

Por ejemplo, si tomamos un elemento de HTML con un ancho específico y establecemos su propiedad `margin` como `auto`, esto le indica al documento que automáticamente coloque márgenes iguales a la izquierda y a la derecha de nuestro elemento, centrándolo en la página.



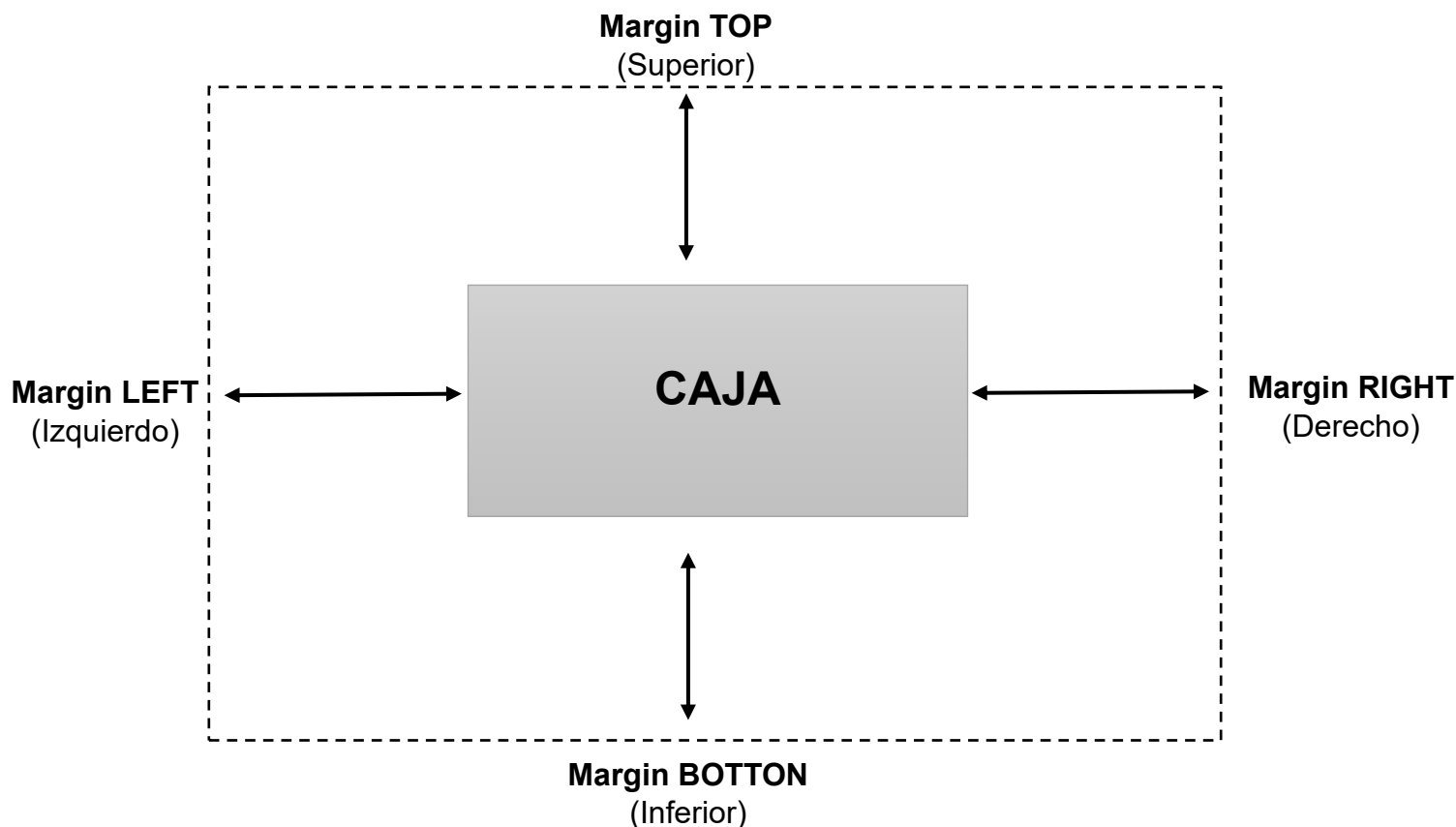
Ejemplo:

```
1  * {
2    border: 1px dashed black;
3  }
4
5  div {
6    height: 50px;
7    width: 100px;
8    border: 2px solid black;
9    border-radius: 5px;
10   background-color: #308014;
11   margin: auto
12 }
```


9.1.2 Margen superior, derecho, inferior, izquierdo

Si quieres establecer un margen determinado, debes hacer esto:

```
1  margin-top: /*algún valor*/  
2  margin-right: /*algún valor*/  
3  
4  margin-bottom: /*algún valor*/  
5  margin-left: /*algún valor*/
```



También es posible establecer al mismo tiempo todos los márgenes de un elemento: simplemente empieza con el margen superior y avanza en el sentido de las agujas del reloj (del superior al derecho, al inferior, y al izquierdo).

Por ejemplo

```
margin: 1px 2px 3px 4px;
```

Establecerá un margen superior de 1 píxel, un margen derecho de 2, un margen inferior de 3 y un margen izquierdo de 4.

Ejercicio práctico

1. Utiliza el siguiente código HTML para el ejercicio.

```
<!DOCTYPE html>
<html>
  <head>
    <link type="text/css" rel="stylesheet"
href="stylesheet.css"/>
    <title>Resultado</title>
  </head>
  <body>
    <div></div>
  </body>
</html>
```

Instrucciones

2. Utiliza el siguiente código CSS para el ejercicio

```
* {
  border: 1px dashed black;
}

div {
  height: 50px;
  width: 100px;
  border: 2px solid black;
  border-radius: 5px;
  background-color: #308014;
  <----- aquí va el código para margen ----->
}
```

3. Usando el método que prefieras, pon un margen superior de 20px, un margen derecho de 50px, un margen inferior de 10px y un margen izquierdo de 5px.

9.2 Padding (relleno)

No olvides: el relleno es el espacio entre tu borde y su capa más interna: el contenido propiamente dicho.

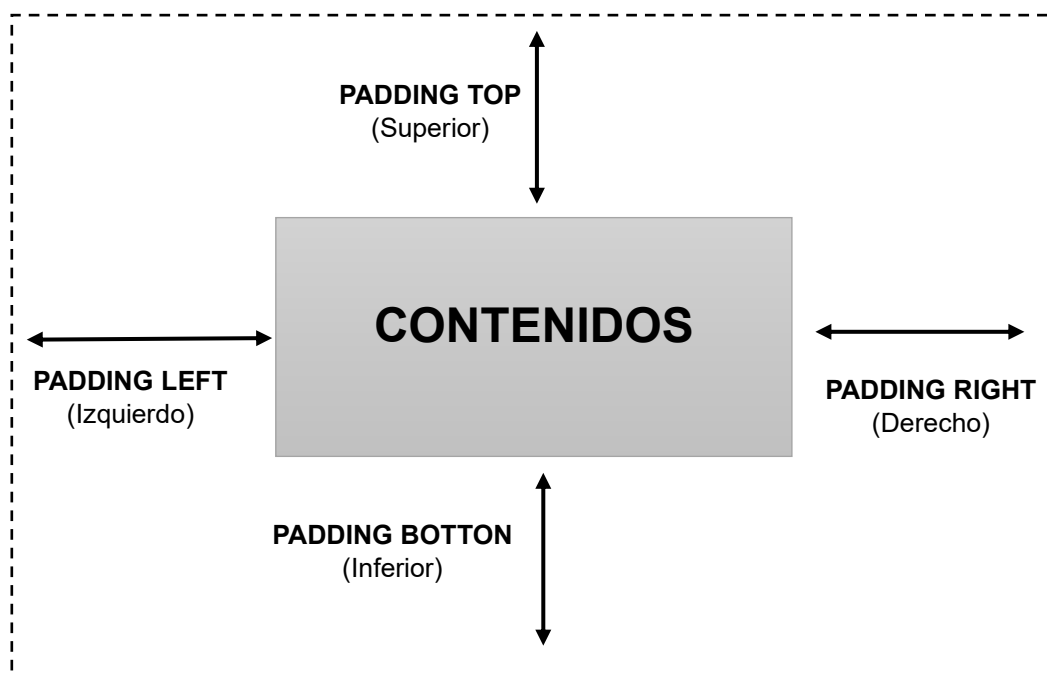
Los rellenos pueden establecerse de dos formas, igual que los márgenes. Podés seleccionarlos uno por uno, así:

```
padding-top: /*algún valor*/
padding-right: /*algún valor*/
padding-bottom: /*algún valor*/
padding-left: /*algún valor*/
```

O pueden ser seleccionados todos en una sola declaración, así:

```
padding: valor valor valor valor;
```

También tienes que saber que si quieres que tu relleno sea el mismo para los cuatro lados, debes declarar ese valor solamente una vez. `padding: 10px` le dará a tu elemento de HTML 10 píxeles de relleno a los cuatro lados.



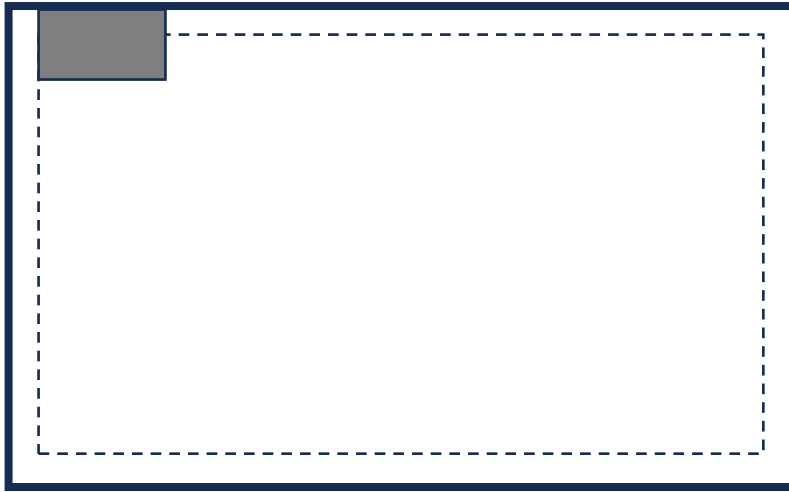
Ejemplo:

```
* {  
  border: 1px dashed black;  
}  
  
div {  
  height: 50px;  
  width: 100px;  
  border: 4px color:#FF0000 dashed;  
  border-radius: 5px;  
  background-color: #308014;  
  padding: 40px;  
}
```

9.2.1 Valores negativos

Cuando le das un valor positivo de relleno o de margen a CSS, CSS coloca ese espacio entre el elemento y su referente: por ejemplo, si tiene una `<div>` y buscas darle un margen izquierdo, `margin-left` de 20px, CSS colocará veinte píxeles entre el margen izquierdo de ese `<div>` y el borde de la pantalla. Esto efectivamente desplaza la `<div>` veinte píxeles a la derecha.

Si quieres desplazar un elemento en la otra dirección, entonces hay que darle a CSS un valor negativo: `margin-left: -20px` desplazará el elemento veinte píxeles a la izquierda.



Ejemplo margin-top: -20px

Ejercicio: Practicar cajas, borders y rellenos.

1. Usaremos este código para el ejercicio

```
<!DOCTYPE html>
<html>
  <head>
    <link type="text/css" rel="stylesheet" href="stylesheet.css"/>
    <title>Resultado</title>
  </head>
  <body>
    <div></div>
  </body>
</html>
```

2. Crea el archivo CSS recuerda que el nombre tiene que ser el mismo que el que usamos en la etiqueta link
3. Escribe el código CSS para hacer lo siguiente:
 - Haz que ese div tenga un borde negro de 1px, de línea continua.
 - Dale color de fondo #CC0000.
 - Establece su margen superior en 10px, su margen derecho en 5px, su margen inferior en 5px y su margen izquierdo en 50px.
 - Establece su relleno (propiedad padding) superior en 0px, su relleno derecho en 30px, su relleno inferior en 0px y su relleno izquierdo en 10px.

10. Floats

¿Cómo determinamos el lugar de la página donde van? Una forma de hacerlo es usar **floats**. Cuando usas float para un elemento de la página, le estás indicando a la página: "Estoy por decirte donde ubicar este elemento, pero tienes que ponerlo dentro del flujo de los demás elementos." Esto quiere decir que si existen varios elementos flotantes, todos saben que los demás están ahí, y no se superponen.

Imagina que la página de HTML es el mar y los elementos flotantes son los barcos: todos los barcos tienen posiciones en el mar, y todos se ven y se mantienen fuera del camino de los demás.

(Algunos de los métodos de posicionamiento que aprenderemos en las secciones siguientes pueden hacer que los elementos se superpongan accidentalmente).

Ejemplo:

```
div {
  height: 300px;
  width: 100px;
  border: 2px solid black;
  border-radius: 5px;
  background-color: #308014;
  Float:right
}
```

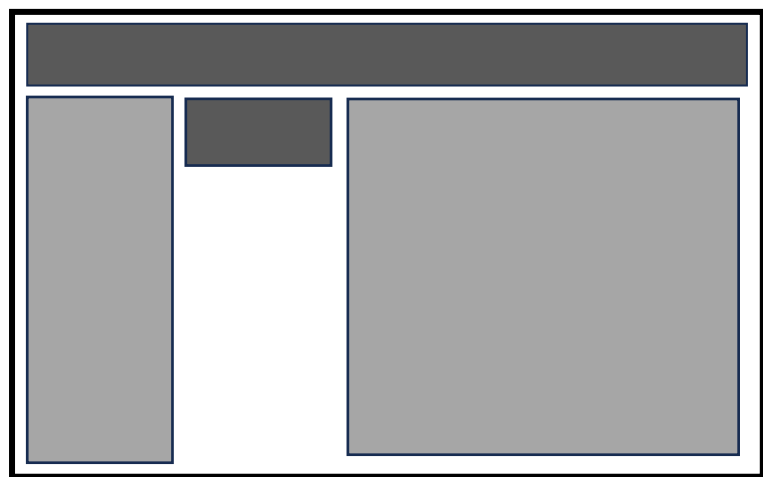
Este código lo puedes poner en práctica si utilizas en código HTLM del ejercicio anterior, si quieres que flote hacia la izquierda cambiar foat:right por float:left. Tambien es posible combinar floats, para que algunos elemento vayan hacia la izquierda y otros hacia la derecha.

Ejemplo:

```
div {
  height: 300px;
  width: 300px;
  border: 2px solid black;
  background-color: #308014;
  Float:right
}

p {
  font-family: Verdana, sans-serif;
  width: 280px;
  Float:left
}
```

Lamentablemente, a veces es posible mezclar elementos grandes, flotantes y no flotantes, y los elementos terminan superponiéndose.



Observa el cuadro pequeño que esta entre los 2 cuadros más grandes. Está atascado ahí atrás porque no se le ha indicado algo importante: ¡que se aleje (clear) de los demás elementos de la página!

Si usas la propiedad `clear:left` para un elemento, este inmediatamente se desplazará debajo de cualquier elemento flotante del lado izquierdo de la página; también se puede usar `right` para desplazarlo hacia el lado derecho. ¡O usar `clear:both` para que se aleje de los elementos que están a la izquierda y a la derecha!. La sintaxis es la misma a la que usamos siempre:

```
elemento {  
    clear: /*right, left, o both*/  
}
```

Ejercicio

1. Usaremos el siguiente código HTML

```
<!DOCTYPE html>  
<html>  
    <head>  
        <link type="text/css" rel="stylesheet" href="stylesheet.css"/>  
        <title>Resultado</title>  
    </head>  
    <body>  
        <div id="encabezado"></div>  
        <div class="izquierda"></div>  
        <div class="derecha"></div>  
        <div id="pieDePagina"></div>  
    </body>  
</html>
```

2. Usaremos este código CSS:

```
div {  
    border-radius: 5px;  
}  
  
#encabezado {  
    height: 50px;  
    background-color: #F38630;  
    margin-bottom: 10px;  
}  
  
.izquierda {  
    height: 300px;  
    width: 150px;  
    background-color: #A7DBD8;  
    float: left;  
    margin-bottom: 10px;  
}  
  
.derecha {  
    height: 300px;  
    width: 450px;  
    background-color: #E0E4CC;  
    float: right;  
    margin-bottom: 10px;  
}  
  
#pieDePagina {  
    height: 50px;  
    background-color: #69D2E7;  
    /*aqui trabajaremos*/  
}
```

3. Haz los cambios necesarios para resolver el problema del cuadro azul que se sobrepone

Establecí la propiedad `clear` del `div` con el identificador `#pieDePagina` (pie de página) como `both`